

Practical week - 5

MySQL Workbench Code:

```
CREATE DATABASE library_db; USE
library_db;

CREATE TABLE IF NOT EXISTS members ( id INT
    AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(100) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role VARCHAR(30) NOT NULL DEFAULT 'member'
);

CREATE TABLE IF NOT EXISTS books (
    id INT AUTO_INCREMENT PRIMARY KEY, title
    VARCHAR(250) NOT NULL,
    author VARCHAR(200) NOT NULL,
    isbn VARCHAR(13) UNIQUE NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    published_date DATE NOT NULL, description
    TEXT,
    embedding JSON
);
```

database.py

```
from sqlalchemy.ext.asyncio import (
    AsyncSession, async_sessionmaker, create_async_engine
)

from sqlalchemy.orm import DeclarativeBase DATABASE_URL = (
    "mysql+asyncmy://root:YOUR_MYSQL_PASSWORD" "@localhost:3306/library_db"
)

engine = create_async_engine(DATABASE_URL, echo=True)
AsyncSessionLocal = async_sessionmaker(
```

```

        engine, expire_on_commit=False
    )
class Base(DeclarativeBase): pass
async def get_db():
    async with AsyncSessionLocal() as session: yield
        session

```

models.py

```

from datetime import date from
decimal import Decimal

from sqlalchemy import Date, Integer, Numeric, String, Text, JSON from
sqlalchemy.orm import Mapped, mapped_column
from database import Base class
Member(Base):
    __tablename__ = "members"
    id: Mapped[int] = mapped_column( Integer,
        primary_key=True
    )
    username: Mapped[str] = mapped_column(
        String(100), unique=True, nullable=False
    )
    password_hash: Mapped[str] = mapped_column( String(255),
        nullable=False
    )
    role: Mapped[str] = mapped_column( String(30),
        default="member", nullable=False
    )
class Book(Base):
    __tablename__ = "books"
    id: Mapped[int] = mapped_column( Integer,
        primary_key=True
    )
    title: Mapped[str] = mapped_column(

```

```

        String(250), nullable=False
    )
    author: Mapped[str] = mapped_column(
        String(200), nullable=False
    )
    isbn: Mapped[str] = mapped_column( String(13),
        unique=True, nullable=False
    )
    price: Mapped[Decimal] = mapped_column(
        Numeric(10, 2), nullable=False
    )
    published_date: Mapped[date] = mapped_column( Date,
        nullable=False
    )
    description: Mapped[str | None] = mapped_column( Text,
        nullable=True
    )
    embedding: Mapped[list[float] | None] = mapped_column( JSON,
        nullable=True
    )

```

schemas.py

```

from datetime import date from
decimal import Decimal
from pydantic import BaseModel, Field, field_validator class
BookSchema(BaseModel):
    title: str
    author: str
    isbn: str = Field(
        min_length=13,
        max_length=13
    )
    price: Decimal = Field(gt=0)

```

```

    published_date: date description: str |
    None = None
    @field_validator("published_date")
    @classmethod
    def validate_published_date(cls, value: date): if value
        > date.today():
            raise ValueError(
                "published_date cannot be in the future"
            )
        return value

class MemberRegister(BaseModel): username: str
    password: str = Field(min_length=6) class
TokenResponse(BaseModel):
    access_token: str
    token_type: str

```

security.py

```

from datetime import datetime, timedelta, timezone from fastapi
import Depends, HTTPException
from fastapi.security import OAuth2PasswordBearer from jose
import JWTError, jwt
from passlib.context import CryptContext from
sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession from
database import get_db
from models import Member SECRET_KEY =
"CHANGE_THIS_SECRET" ALGORITHM =
"HS256" ACCESS_TOKEN_EXPIRE_MINUTES = 60
pwd_context = CryptContext(
    schemes=["bcrypt"],
    deprecated="auto"
)
oauth2_scheme = OAuth2PasswordBearer(

```

```

        tokenUrl="/login"
    )
def hash_password(password: str): return
    pwd_context.hash(password)
def verify_password(
    plain_password,
    hashed_password
):
    return pwd_context.verify(
        plain_password, hashed_password
    )
def create_access_token(username, role):
    expire = datetime.now(timezone.utc) + timedelta(
        minutes=ACCESS_TOKEN_EXPIRE_MINUTES
    )
    payload = {
        "sub": username,
        "role": role, "exp":
        expire
    }
    return jwt.encode(
        payload, SECRET_KEY,
        algorithm=ALGORITHM
    )
async def get_current_member(
    token: str = Depends(oauth2_scheme), db:
    AsyncSession = Depends(get_db)
):
    credentials_exception = HTTPException(
        status_code=401,
        detail="Invalid or expired token", headers={
            "WWW-Authenticate": "Bearer"
        }
    )

```

```

)
try:
    payload = jwt.decode(
        token,
        SECRET_KEY,
        algorithms=[ALGORITHM]
    )
    username = payload.get("sub") if not
    username:
        raise credentials_exception
except JWTError:
    raise credentials_exception
result = await db.execute(
    select(Member).where( Member.username
        == username
    )
)
member = result.scalar_one_or_none() if not
member:
    raise credentials_exception
return member

```

embedding.py

```

import hashlib import
math VECTOR_DIM =
384
def text_to_vector(text: str) -> list[float]: values = []
    for i in range(VECTOR_DIM): digest =
        hashlib.sha256(
            f"{text.lower()}:{i}".encode()
        ).digest()
        number = int.from_bytes(
            digest[:4], byteorder="big"
        )

```

```

        values.append(
            (number / 2**32) * 2 - 1
        )
    magnitude = math.sqrt( sum(x * x
        for x in values)
    )
    return [
        x / magnitude
        for x in values
    ]

```

main.py

```

from fastapi import (
    BackgroundTasks,
    Depends,
    FastAPI,
    HTTPException,
    Query,
    status
)
from fastapi.security import OAuth2PasswordRequestForm
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from database import Base, engine, get_db
from models import Book, Member
from schemas import BookSchema, MemberRegister, TokenResponse
from security import (
    create_access_token,
    get_current_member,
    hash_password, verify_password
)
from embedding import text_to_vector
import math

app = FastAPI(
    title="Library Backend & Security API",
    version="1.0"
)

# Create tables when application starts
@app.on_event("startup")
async def startup():
    async with engine.begin() as connection:
        await connection.run_sync(
            Base.metadata.create_all

```

```

    )
# Home @app.get("/")
async def home():
    return {
        "message": "Library API is running"
    }
# Background task for logging
def log_new_book(book_id: int, title: str): with
    open(
        "book_events.txt", "a",
        encoding="utf-8"
    ) as file:
        file.write(
            f"New Book Added: id={book_id}, "
            f"title={title}\n"
        )
# Cosine similarity def
cosine_similarity(
    a: list[float],
    b: list[float]
) -> float:
    if len(a) != len(b): return -
        1.0
    dot = sum( x
        * y
        for x, y in zip(a, b)
    )
    norm_a = math.sqrt( sum(x *
        x for x in a)
    )
    norm_b = math.sqrt( sum(y *
        y for y in b)
    )
    if norm_a == 0 or norm_b == 0: return 0.0
    return dot / (norm_a * norm_b) #
Register member @app.post("/register")
async def register(
    member: MemberRegister,
    db: AsyncSession = Depends(get_db)
):
    result = await db.execute(
        select(Member).where(
            Member.username == member.username
        )
    )
    if result.scalar_one_or_none(): raise
        HTTPException(
            status_code=400, detail="Username

```

```

        already exists"
    )
    new_member = Member(
        username=member.username,
        password_hash=hash_password(
            member.password
        ),
        role="member"
    )
    db.add(new_member) await
    db.commit()
    await db.refresh(new_member)
    return {
        "message": "Member registered successfully", "username":
        new_member.username,
        "role": new_member.role
    }
# Login
@app.post(
    "/login", response_model=TokenResponse
)
async def login(
    form_data: OAuth2PasswordRequestForm = Depends(), db:
    AsyncSession = Depends(get_db)
):
    result = await db.execute(
        select(Member).where(
            Member.username == form_data.username
        )
    )
    member = result.scalar_one_or_none() if (
        not member
        or not verify_password( form_data.password,
            member.password_hash
        )
    ):
        raise HTTPException(
            status_code=401,
            detail="Invalid username or password"
        )
    token = create_access_token(
        member.username, member.role
    )
    return {
        "access_token": token,
        "token_type": "bearer"
    }
# Add book - protected by JWT @app.post(

```

```

    "/books",
    status_code=status.HTTP_201_CREATED
)
async def create_book( book:
    BookSchema,
    background_tasks: BackgroundTasks, current_member:
    Member = Depends(
        get_current_member
    ),

    db: AsyncSession = Depends(get_db)
):
    # Create embedding embedding =
    text_to_vector(
        f"{book.title} "
        f"{book.author} "
        f"{book.description or ""}
    )
    new_book = Book(
        title=book.title,
        author=book.author,
        isbn=book.isbn,
        price=book.price,
        published_date=book.published_date,
        description=book.description,
        embedding=embedding
    )
    db.add(new_book)
    await db.commit()
    await db.refresh(new_book)
    # Schedule background logging
    background_tasks.add_task(
        log_new_book, new_book.id,
        new_book.title
    )
    return {
        "message": "Book added successfully", "book_id":
        new_book.id,
        "added_by": current_member.username
    }
# Search books using cosine similarity
@app.get("/books/search")
async def search_books(
    q: str = Query(min_length=1), limit:
    int = Query(
        default=5,
        ge=1, le=20
    ),
    current_member: Member = Depends(

```

```

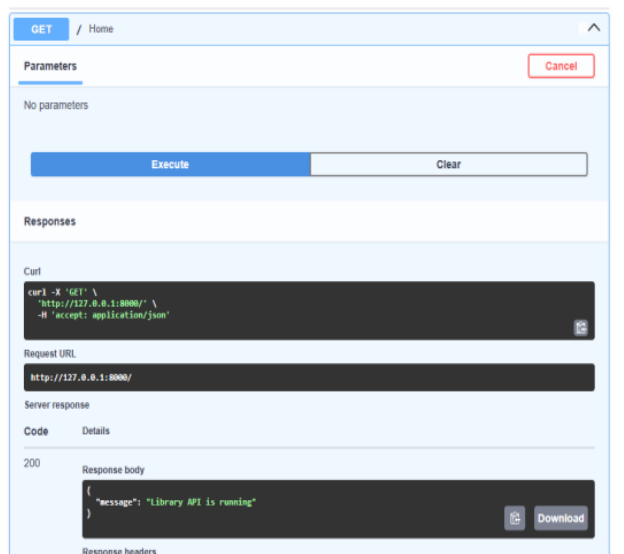
        get_current_member
    ),
    db: AsyncSession = Depends(get_db)
):
    query_vector = text_to_vector(q) result =
    await db.execute(
        select(Book).where(
            Book.embedding.is_not(None)
        )
    )

    books = result.scalars().all() ranked =
    []
    for book in books:
        if book.embedding:

            similarity = cosine_similarity(
                query_vector, book.embedding
            )
            ranked.append({ "id":
                book.id, "title":
                book.title,
                "author": book.author,
                "isbn": book.isbn, "price":
                float(book.price),
                "published_date":
                book.published_date, "description":
                book.description,
                "similarity":
                similarity
            })
    ranked.sort(
        key=lambda item: item["similarity"],
        reverse=True
    )
    return {
        "query": q,
        "results": ranked[:limit]
    }

```

GET:



1. Register

POST /register



After execution:

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/register' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "username": "sujit",
    "password": "Sujit@2026/"
  }'
```

Request URL

http://127.0.0.1:8000/register

Server response

Code	Details
------	---------

200	
-----	--

Response body

```
{
  "message": "Member registered successfully",
  "username": "sujit",
  "role": "member"
}
```

Response headers

```
content-length: 86
content-type: application/json
date: Sat, 13 Sep 2025 05:26:35 GMT
server: uvicorn
```

Authorization:

Available authorizations ✕

Scopes are used to grant an application different levels of access to data on behalf of the end user. Each API may declare one or more scopes.
API requires the following scopes. Select which ones you want to grant to Swagger UI.

OAuth2PasswordBearer (OAuth2, password)

Authorized

Token URL: /login
Flow: password
username: sujit
password: Sujit@2026\
Client credentials location: basic
client_secret: *****

LogoutClose

2. Login

POST /login

Response:

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/login' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  -d 'grant_type=password&username=Priyanka2026&password=priyanka123&scope=&client_id=string&client_secret=*****'
```

Request URL

```
http://127.0.0.1:8000/login
```

Server response

Code	Details
200	Response body

POST /login Login

Parameters

No parameters

Request body required

grant_type string (string null) <i>pattern: ^password\$</i>	password ☐ Send empty value
username * required string	sujit
password * required string(\$password)	Sujit@2026\
scope string	scope ☒ Send empty value
client_id string (string null)	string ☐ Send empty value
client_secret string (string null)(\$password)	***** ☐ Send empty value

3. POST /books

POST

/books Create Book



Parameters

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "title": "sujit's Guide to Java",
  "author": "sujit",
  "isbn": "978123456789",
  "price": 599.99,
  "published_date": "2025-08-15",
  "description": "A beginner friendly guide to Java programming and data structures."
}
```

Response:

Responses

Curl

```
curl -X 'POST' \
"http://127.0.0.1:8080/books" \
-H 'accept: application/json' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpzZW50L3VudC9ybnNpdjEwMTQwZmFjaW50eSIsImVudCI6IjE2ZDmJfjoKcgONT3MnEThIQk.BdA7hgWuNRdUv1KISmoOY27hw0-1tZhdvuutGrFSQ9UD' \
-H 'Content-Type: application/json' \
-d '{
"title": "sujit's Guide to Java",
"author": "sujit",
"isbn": "978123456789",
"price": "599.99",
"published_date": "2025-08-15",
"description": "A beginner friendly guide to Java programming and data structures."
}'
```

Request URL

```
http://127.0.0.1:8080/books
```

Server response

CodeDetails

201Response body

```
{
  "message": "Book added successfully",
  "book_id": 1,
  "added_by": "sujit"
}
```

Response headers

```
content-length: 75
content-type: application/json
date: Sat, 12 Sep 2025 05:32:53 GMT
server: uvicorn
```

4. GET /books/search

GET

/books/search

Search Books

Parameters

Cancel

Name	Description
q * required string (query)	Java minlength: 1
limit integer (query)	5 maximum: 20 minimum: 1

Execute

Clear

Response:

5. Test validation

Test A — Invalid ISBN

POST

/books

Create Book

Parameters

Cancel

Reset

No parameters

Request body

required

application/json

Edit Value

Schema

```
{
  "title": "Test Book",
  "author": "Test Author",
  "isbn": "12345",
  "price": 100,
  "published_date": "2024-01-01",
  "description": "Test"
}
```

Response will be 422 error

Server response

Code

Details

422

Error: Unprocessable Entity

Response body

```
{
  "detail": [
    {
      "type": "string_too_short",
      "loc": [
        "body",
        "isbn"
      ],
      "msg": "String should have at least 13 characters",
      "input": "12345",
      "ctx": {
        "min_length": 13
      }
    }
  ]
}
```

Download

Test B — Negative price

POST

/books

Create Book

Parameters

Cancel

Reset

No parameters

Request body

required

application/json

Edit Value

Schema

```
{
  "title": "Test Book",
  "author": "Test Author",
  "isbn": "1234567890100",
  "price": -100,
  "published_date": "2024-01-01",
  "description": "Test"
}
```

Response will be 422 because price<0

Code	Details
422	Error: Unprocessable Entity Response body <pre>{ "detail": [{ "type": "greater_than", "loc": ["body", "price"], "msg": "Input should be greater than 0", "input": -100, "ctx": { "gt": 0 } }] }</pre>  Download

Test C — Future publication date

POST /books Create Book

Parameters

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "title": "Future Book",
  "author": "Test Author",
  "isbn": "1234567890124",
  "price": 100,
  "published_date": "2035-01-01",
  "description": "Test"
}
```

Response will be 422 error because a book cannot have a future publication date

Code	Details
422	Error: Unprocessable Entity Response body <pre>{ "detail": [{ "type": "value_error", "loc": ["body", "published_date"], "msg": "Value error, published_date cannot be in the future", "input": "2035-01-01", "ctx": { "error": {} } }] }</pre>  Download